Reporte del Proyecto GM

Videojuego Recolectar Gotas

Asignatura	INF2236 - Programación Avanzada
Profesor	Claudio Cubillos
Período	2do Semestre 2026
Integrantes	Nicolas Jaramillo, Lucas Ahumada, Alessandro Reginato
Fecha de entrega	13/11/2026

Resumen del proyecto

Juego base: GameLluviaMenu2024

Tecnologías utilizadas: Java 11, LibGDX 1.12.1, Gradle, GitHub y NetBeans

Forma de ejecución: aplicación de escritorio mediante el módulo lwjgl3

Tabla de Contenidos

Sección	Página
GM-1 Explicación del juego desde la perspectiva del jugador	2-4
GM-2 Análisis del juego desde la perspectiva de ingeniería de software	5-6
GM-3 Diseño de diagrama UML con clases del Dominio del juego	7-8
GM-4 Diseño y codificación de clase abstracta e interfaz	9
GM-5 Aplicación de encapsulamiento y principios de orientación a objetos	10
GM-6 Aplicación del patrón de diseño Singleton	11
GM-7 Aplicación del patrón de diseño Strategy	12
GM-8 Aplicación del patrón de diseño Template Method	13
GM-9 Aplicación del patrón de diseño Builder	14
GM-10 Utilización de GitHub	15

GM-1 Explicación del juego desde la perspectiva del jugador

Requisito solicitado: Explicación del juego desde la perspectiva del jugador, incluyendo objetivos, narrativa, elementos y controles, utilizando maquetas o pantallazos.

Objetivos

Nuestro juego consiste en recolectar gotas buenas y evitar las gotas dañinas utilizando un tarro. El objetivo es conseguir la mayor cantidad de puntos antes de perder todas las vidas.

El jugador puede elegir entre el modo fácil y el modo difícil. En el modo fácil comienza con tres vidas y las gotas caen con menor frecuencia. En el modo difícil comienza con dos vidas y las gotas caen más seguido. Cada gota buena recolectada suma 10 puntos, mientras que cada gota dañina que toca el tarro hace perder una vida.

Narrativa

La historia comienza en Chile en 2026, durante una tormenta ficticia inspirada en El Niño. Entre las gotas rojas todavía cae agua limpia. El jugador tiene un tarro y debe rescatar toda el agua que pueda porque el mundo la necesita para sobrevivir. Al abrir el juego se muestran tres frases que aparecen y desaparecen suavemente con el sonido de la lluvia. Después se abre el menú de dificultad.

Elementos

Tarro: es el elemento que controla el jugador para recolectar las gotas. Se mueve horizontalmente por la parte inferior de la pantalla.

Gotas buenas: al recolectarlas, suman 10 puntos.

Gotas dañinas: al tocar el tarro, hacen perder una vida.

Puntaje: muestra los puntos acumulados durante la partida. En la pantalla aparece con el nombre “Gotas totales”.

Vidas: indica cuántas vidas le quedan al jugador. El modo fácil comienza con tres y el modo difícil con dos.

HighScore: muestra el mejor puntaje del modo elegido mientras el juego está abierto. Cada modo tiene su propio récord.

Modos de juego: en el modo fácil aparecen gotas con menor frecuencia y en el modo difícil con mayor frecuencia, además de tener una vida menos. El modo difícil también usa un fondo rojo.

Pantallas: al abrir el juego aparece la intro. Después vienen el menú principal, la partida, la pantalla de pausa y la pantalla de fin del juego.

Sonidos: se escucha lluvia de fondo y efectos al recolectar gotas buenas o recibir daño.

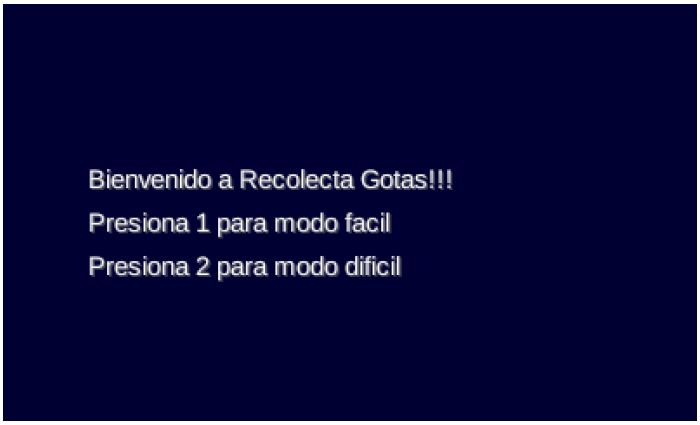
Controles

La introducción se puede saltar con Enter. En el menú, la tecla 1 inicia el modo fácil y la tecla 2 inicia el difícil.

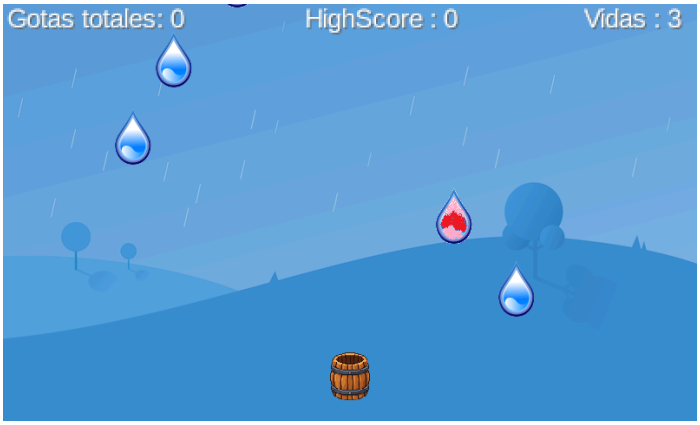
Al empezar la partida, el tarro se mueve con las flechas izquierda y derecha. Si el jugador presiona 2, puede moverlo manteniendo presionado el botón del mouse y desplazando el puntero. Con la tecla 1 vuelve al teclado. Para pausar el juego se presiona Escape y para reanudar el juego basta con hacer click en la pantalla de pausa. Cuando se acaban las vidas, la tecla 1 inicia otra partida en la misma dificultad y la tecla 2 vuelve al menú.



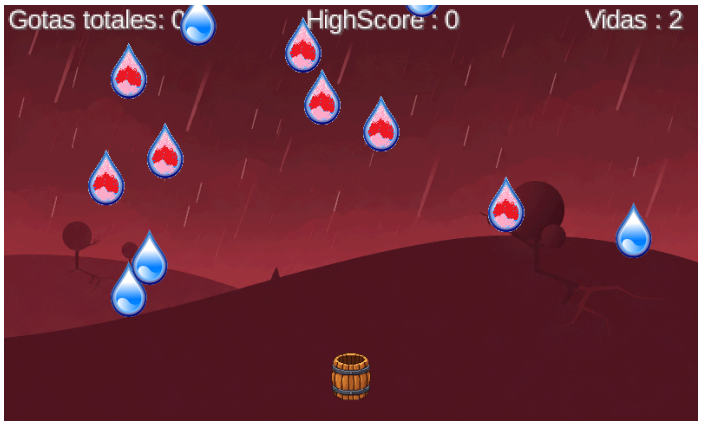
Intro. La historia presenta la tormenta en Chile antes de elegir la dificultad.



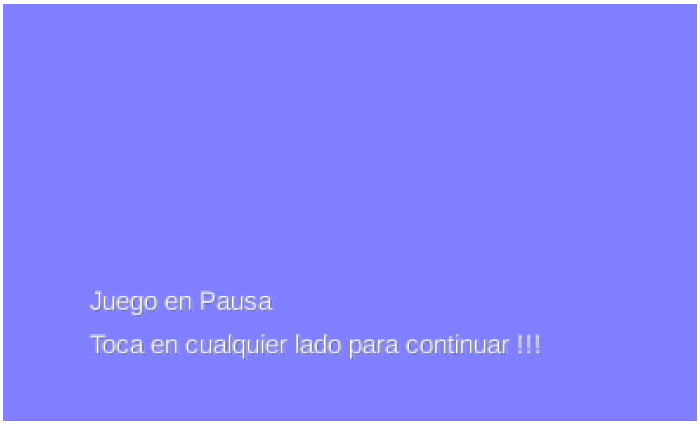
Menú principal. Se puede elegir el modo fácil con 1 o el difícil con 2.



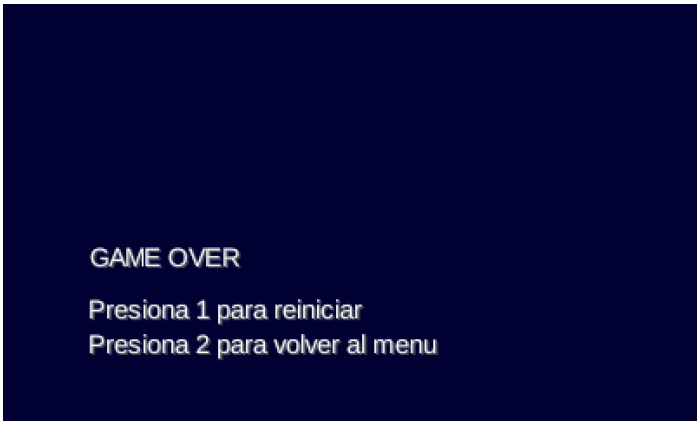
Modo fácil. La partida comienza con tres vidas y las gotas aparecen con menos frecuencia.



Modo difícil. El fondo es rojo, las gotas aparecen más seguido y el jugador empieza con dos vidas.



Pantalla de pausa. Se abre con Esc y se puede retomar la partida con un clic.



Fin de la partida. Se puede reiniciar con 1 o volver al menú con 2.

GM-2 Análisis del juego desde la perspectiva de ingeniería de software

Requisito solicitado: Análisis del juego desde la perspectiva de ingeniería de software, especificando el juego base y las modificaciones propuestas.

Modificaciones propuestas

Juego base utilizado

El proyecto utiliza como base **GameLluviaMenu2024**, desarrollado en Java con LibGDX, según el README del repositorio. Es un juego de recolección en el que el jugador controla un tarro y debe capturar gotas beneficiosas mientras evita las dañinas. El proyecto utiliza Java 11 y Gradle. README.

Funcionamiento del juego base

El tarro se desplaza horizontalmente mediante las flechas izquierda y derecha, manteniéndose dentro de los límites de la pantalla. Cada partida comienza con tres vidas y cero puntos.

Las gotas aparecen en posiciones horizontales aleatorias y caen hacia la parte inferior de la pantalla. Al recoger una gota buena, el jugador obtiene diez puntos; al tocar una dañina, pierde una vida. Las gotas que salen por la parte inferior se eliminan sin modificar el puntaje ni las vidas.

Cuando se agotan las vidas, se muestra la pantalla de fin de partida y se permite reiniciar. El juego también dispone de menú inicial, pantalla de pausa, efectos de sonido y música de lluvia. El puntaje máximo se conserva entre partidas durante la ejecución de la aplicación, pero no se guarda en un archivo. Este comportamiento se encuentra en Tarro, Lluvia, GameScreen, GameLluviaMenu y las clases de pantallas.

Análisis de la estructura original

En la versión inicial, Lluvia representaba las gotas mediante dos colecciones paralelas: rainDropsPos, que almacenaba rectángulos con sus posiciones, y rainDropsType, que almacenaba números para identificar su tipo.

Esta organización exigía mantener ambas colecciones sincronizadas. Además, Lluvia debía comprobar el tipo de cada gota para decidir qué imagen dibujar y qué efecto aplicar al tarro. Esto concentraba varias responsabilidades en una misma clase y dificultaba incorporar nuevos comportamientos. Código inicial de Lluvia.

Modificaciones propuestas y avance actual

La versión final reorganiza las gotas mediante abstracción, herencia y polimorfismo.

- La clase abstracta Gota reúne la posición, imagen, área de colisión y movimiento compartidos.
- GotaBuena y GotaMala heredan de Gota y definen su efecto sobre el tarro.
- La interfaz ComportamientoGota establece las operaciones de movimiento, dibujo, consulta de colisiones y salida de pantalla, junto con aplicación del efecto.
- Lluvia utiliza una colección de ComportamientoGota, sustituyendo las dos colecciones paralelas.

Lluvia administra las gotas activas, determina cuándo aparece una nueva y elige su posición horizontal. Para crearla, llama a CreadorGotas. La implementación CreadorGotasAleatorio selecciona el tipo e instancia GotaBuena o GotaMala. Cuando una gota choca con el tarro, Lluvia llama a aplicarEfecto y se suman puntos o quita una vida.

La clase PantallaBase define el ciclo común de dibujo y actualización, usamos IntroScreen para presenta la historia, MainMenuScreen permite elegir la dificultad, GameScreen muestra y controla la partida, PausaScreen permite retomarla y GameOverScreen ofrece reiniciar o volver al menú.

En DirectorPartida se hace uso de PartidaBuilder para preparar el tarro, la lluvia y el fondo. PartidaFacilBuilder y PartidaDificilBuilder configuran los valores determinados para la dificultad. La Partida resultante guarda esos elementos y coordina la actualización del tarro y la lluvia.

La clase Tarro conserva las vidas, puntos y posición, el movimiento del tarro se encuentra en EstrategiaMovimiento, con GameScreen seleccionando el control por teclado o mouse durante la partida. Puntaje conserva en memoria un récord por dificultad y lo actualiza cuando termina una partida con un puntaje superior.

Partes que se mantienen y partes que cambian

Se mantiene del juego base	Se modifica o incorpora
Movimiento horizontal del tarro mediante las flechas.	Representación de cada gota mediante un objeto.
Diez puntos por cada gota buena recolectada.	Clase abstracta Gota.
Gotas buenas y dañinas.	Clases GotaBuena y GotaMala.
Caída de gotas y detección de colisiones.	Interfaz ComportamientoGota.
Menú, pausa, fin de partida y reinicio.	Colección única de gotas en Lluvia.
Imágenes, sonidos y música existentes.	Distribución del comportamiento específico entre las clases de gotas.
	Dos dificultades: facil y dificil, con diferencias en vidas y frecuencia de gotas
	Control mediante mouse o teclado, seleccionable durante la partida

GM-3 Diseño de diagrama UML con clases del Dominio del juego

Requisito solicitado: Diseño de diagrama UML con clases del Dominio del juego, incluyendo clases base LibGDX y propias, y su código en Java.

Clases del dominio y relaciones

Lwjgl3Launcher inicia la aplicación de escritorio y crea GameLluviaMenu. GameLluviaMenu hereda de Game de LibGDX, implementa ContextoJuego y proporciona los recursos de dibujo compartidos.

PantallaBase implementa Screen de LibGDX y utiliza ContextoJuego para acceder a esos recursos y cambiar de pantalla. IntroScreen, MainMenuScreen, GameScreen, PausaScreen y GameOverScreen heredan de PantallaBase. PausaScreen guarda como Screen la pantalla de partida que debe restaurar.

DirectorPartida coordina la construcción mediante PartidaBuilder. PartidaFacilBuilder y PartidaDificilBuilder implementan esa interfaz y preparan una Partida con su tarro, lluvia, fondo y dificultad. Partida mantiene las referencias al tarro y a la lluvia y coordina su actualización. GameScreen solicita la liberación de los recursos al terminar la partida.

Tarro mantiene sus vidas, puntos y posición. Implementa ReceptorGota y delega su movimiento en EstrategiaMovimiento, cuyas implementaciones son MovimientoTeclado y MovimientoTactil.

Lluvia administra una colección de ComportamientoGota y recibe un CreadorGotas. CreadorGotasAleatorio implementa esa interfaz y crea gotas buenas o dañinas. La clase abstracta Gota implementa ComportamientoGota. GotaBuena y GotaMala heredan de Gota y aplican su efecto mediante ReceptorGota.

Puntaje conserva en una instancia compartida el récord de cada dificultad. GameScreen consulta ese récord al dibujar y registra el puntaje final cuando se agotan las vidas.

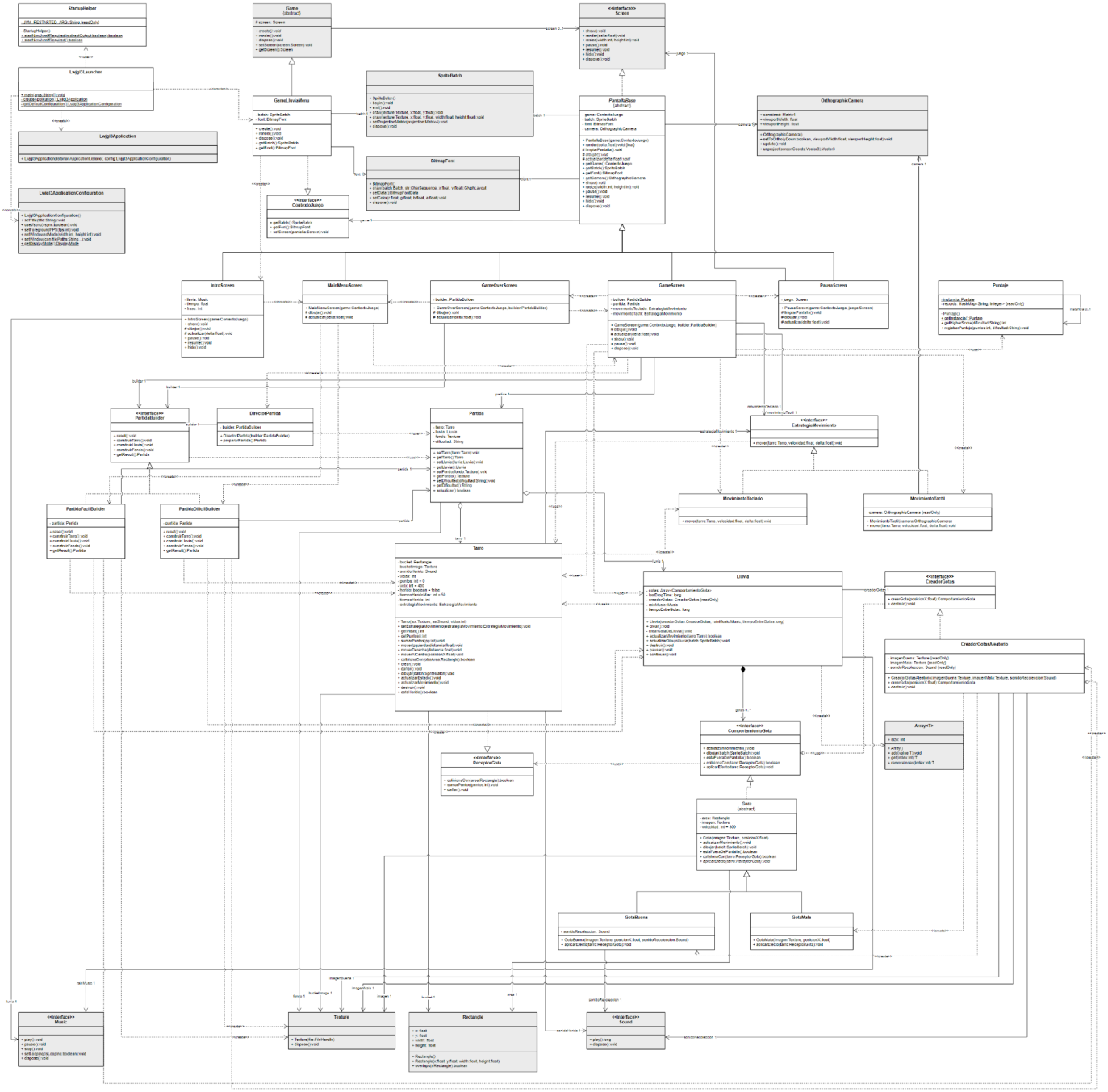
Justificación

Las clases que están en blanco son las propias del proyecto y las grises son de LibGDX. De esta última, incluimos todas las que el proyecto usa de una forma u otra (Game, Screen, SpriteBatch, Texture, Sound, etc…) y omitimos el resto que solo fueron usadas como utilidades estáticas (cabe mencionar que Vector3 no sale en el diagrama UML porque su único uso fue como variable local en un método).

Usamos las interfaces ContextoJuego, ReceptorGota y CreadorGotas para que las pantallas no dependan de GameLluviaMenu, las gotas no dependan de Tarro y Lluvia no tenga que cambiar al agregar un tipo de gota.

Entre Partida y sus partes (Tarro y Lluvia) usamos agregación, porque Partida las recibe del builder y no controla su creación ni su destrucción.

Diagrama UML



Evidencia de implementación

El diagrama muestra las clases propias del juego y las clases base de LibGDX, con sus atributos, métodos y relaciones. Cada clase del diagrama está en su propio archivo .java dentro de core/src/main/java/puppy/code.

GM-4 Diseño y codificación de clase abstracta e interfaz

Requisito solicitado: Diseño y codificación de 1 clase abstracta que sea padre de al menos 2 clases y sea utilizada en el programa, y 1 interfaz que sea implementada por al menos 2 clases y utilizada en el programa.

Clase abstracta

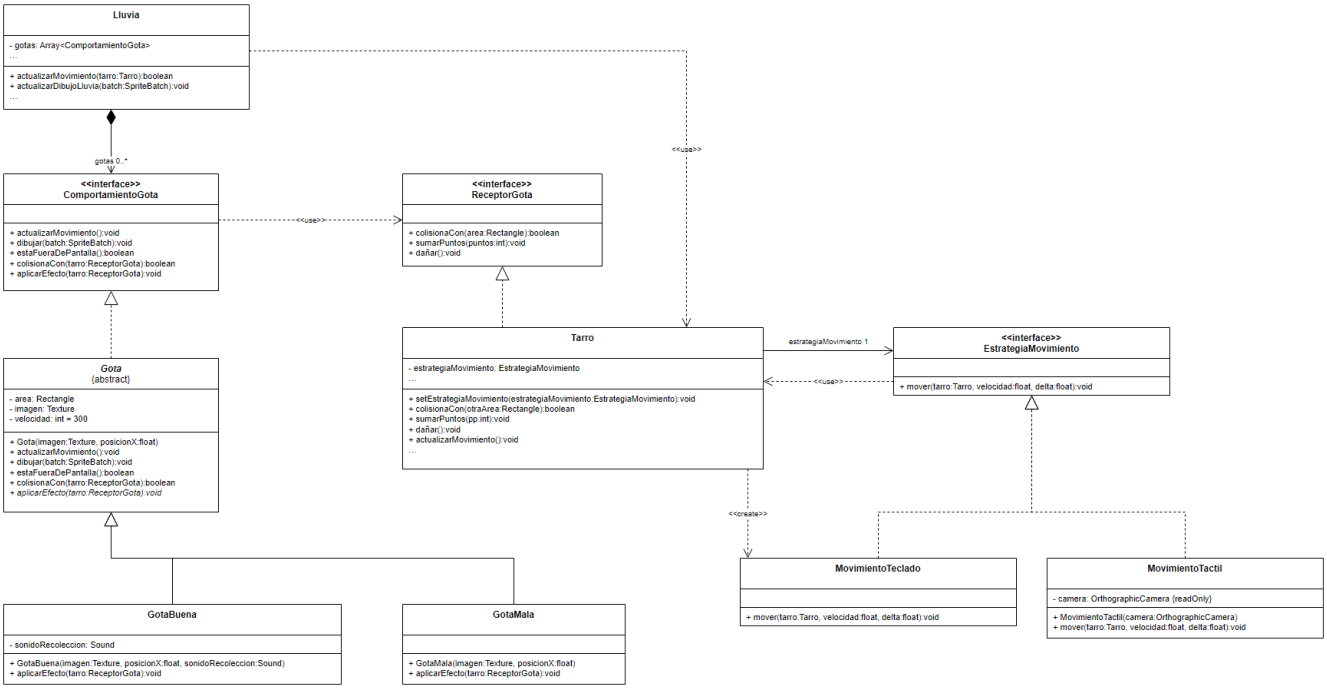
Las gotas buenas y las malas caen, se dibujan y detectan choques de la misma manera, así que dejamos ese código en la clase abstracta Gota. Las dos heredan de ella y definen su propio aplicarEfecto, cuando una GotaBuena toca el tarro, suma 10 puntos y reproduce un sonido, mientras que una GotaMala le quita una vida.

Interfaz

EstrategiaMovimiento es la interfaz que implementan directamente dos clases del proyecto. Define el método mover, que MovimientoTeclado ejecuta a partir de las flechas y MovimientoTactil a partir de la posición del puntero. Tarro usa esa interfaz para delegar su movimiento en la estrategia activa.

Uso en el programa

Lluvia guarda las gotas como elementos de tipo ComportamientoGota. Así puede moverlas, mostrarlas y revisar sus choques usando las mismas acciones para ambos tipos. Cuando una gota toca el tarro, se llama a aplicarEfecto y se ejecuta el efecto de la gota buena o de la mala. Esta organización permite que otro tipo de gota herede el movimiento y el dibujo de Gota y defina su propio efecto. La clase abstracta y su uso se pueden revisar en Gota.java, GotaBuena.java, GotaMala.java, ComportamientoGota.java, ReceptorGota.java y Lluvia.java. La interfaz con dos implementaciones directas se puede revisar en EstrategiaMovimiento.java, MovimientoTeclado.java, MovimientoTactil.java y Tarro.java.



GM-5 Aplicación de encapsulamiento y principios de orientación a objetos

Requisito solicitado: Aplicación de encapsulamiento y principios de orientación a objetos, incluyendo GRASP y SOLID.

Encapsulamiento

Los atributos de nuestras clases son privados y se consultan o modifican mediante métodos públicos. En Tarro, los datos como vidas, puntos, estado herido y posición se mantienen dentro de la clase. Las estrategias de movimiento utilizan moverIzquierda, moverDerecha y moverAlCentro, evitando modificar directamente su rectángulo interno. Gota también mantiene su área privada y permite consultar si salió de la pantalla o si colisiona con el tarro.

Principios GRASP

Experto en Información:

Puntaje conoce el récord y realiza su comparación mediante registrarPuntaje. Partida contiene al tarro y la lluvia, por lo que su método actualizar coordina la actualización de ambos. Tarro y Gota utilizan sus propios datos para comprobar las colisiones.

Alta Cohesion:

organizamos las operaciones según las responsabilidades de cada clase. Partida coordina la actualización del juego y GameScreen mantiene la selección de controles y los cambios de pantalla. En Tarro se separó actualizarEstado de dibujar, de modo que el tiempo herido se actualiza sin depender de la ejecución del dibujo.

Polimorfismo:

Tarro utiliza la interfaz EstrategiaMovimiento para solicitar el movimiento. Según la estrategia asignada, se ejecuta la implementación de MovimientoTeclado o MovimientoTactil, sin que el tarro tenga que comprobar cuál de las dos está utilizando.

Principios SOLID

Abierto/Cerrado:

Tarro puede usar otra forma de control mediante una nueva implementación de EstrategiaMovimiento. Lluvia recibe un CreadorGotas, lo que permite suministrar otra forma de crear gotas sin modificar el ciclo que las actualiza y dibuja. DirectorPartida puede preparar otra dificultad mediante una implementación adicional de PartidaBuilder. Puntaje guarda los récords en un HashMap privado con la dificultad como clave, de modo que una dificultad adicional puede registrar su récord mediante los mismos métodos.

Sustitución de Liskov:

GotaBuena y GotaMala se pueden usar donde se espera una Gota o un ComportamientoGota. Ambas conservan las operaciones de movimiento, dibujo y detección de colisiones. Cada una implementa el efecto que le corresponde cuando ocurre una colisión.

Inversión de dependencias:

PantallaBase obtiene los recursos de dibujo y solicita cambios de pantalla mediante ContextoJuego, la cual es implementada por GameLluviaMenu. Tanto Gota como ComportamientoGota usan ReceptorGota para detectar colisiones y aplicar efectos, con Tarro implementando la interfaz.

GM-6 Aplicación del patrón de diseño Singleton

Requisito solicitado: Aplicación del patrón de diseño Singleton, describiendo problema, contexto, solución y UML.

Problema

En la versión inicial, el puntaje más alto se guardaba en GameLluviaMenu, la clase que inicia el juego, entrega los recursos para dibujar y cambia entre pantallas. GameScreen tenía que pedirle el récord a esa clase para mostrarlo, y al terminar la partida era la misma pantalla la que comparaba los puntos y lo reemplazaba con setHigherScore. Como ese método era público, cualquier clase con acceso al objeto del juego podía reemplazar el récord sin comprobar si el nuevo puntaje era mayor.

Contexto

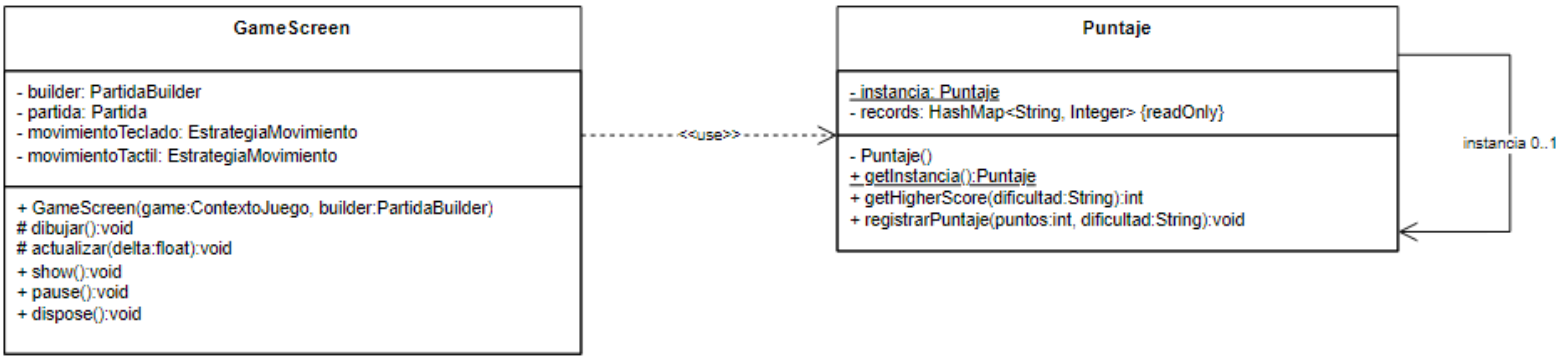
Cada nueva partida crea una instancia de GameScreen. Cuando se agotan las vidas, esa pantalla registra el puntaje y da paso a GameOverScreen. Los récords deben conservarse entre partidas durante la ejecución de la aplicación. Para ello, todas las partidas comparten una única instancia de Puntaje, que almacena un récord separado para cada dificultad.

Solución

Creamos la clase Puntaje aplicando el patrón Singleton. Su constructor es privado, así que ninguna otra clase puede crear un objeto Puntaje. La clase guarda su única instancia en el atributo estático instancia, y el método estático getInstancia la crea la primera vez que se pide y la entrega en las llamadas siguientes. De esta forma, todas las partidas usan el mismo objeto y el récord se conserva mientras el juego está abierto.

Sacamos el récord y sus métodos de GameLluviaMenu, que quedó encargándose solo de los recursos y las pantallas. GameScreen usa getInstancia en dos momentos: al dibujar, para mostrar el récord del modo elegido con getHigherScore, y al terminar la partida, para registrar los puntos con registrarPuntaje. Este último método compara los puntos con el récord del modo jugado y solo lo reemplaza si son mayores, por lo que ninguna clase puede modificarlo directamente.

En el patrón, Puntaje corresponde a la clase Singleton, instancia es la referencia a su único objeto y getInstancia es el punto de acceso que utiliza GameScreen. La evidencia está en: Puntaje.java y GameScreen.java



GM-7 Aplicación del patrón de diseño Strategy

Requisito solicitado: Aplicación del patrón de diseño Strategy, describiendo problema, contexto, solución y UML.

Problema

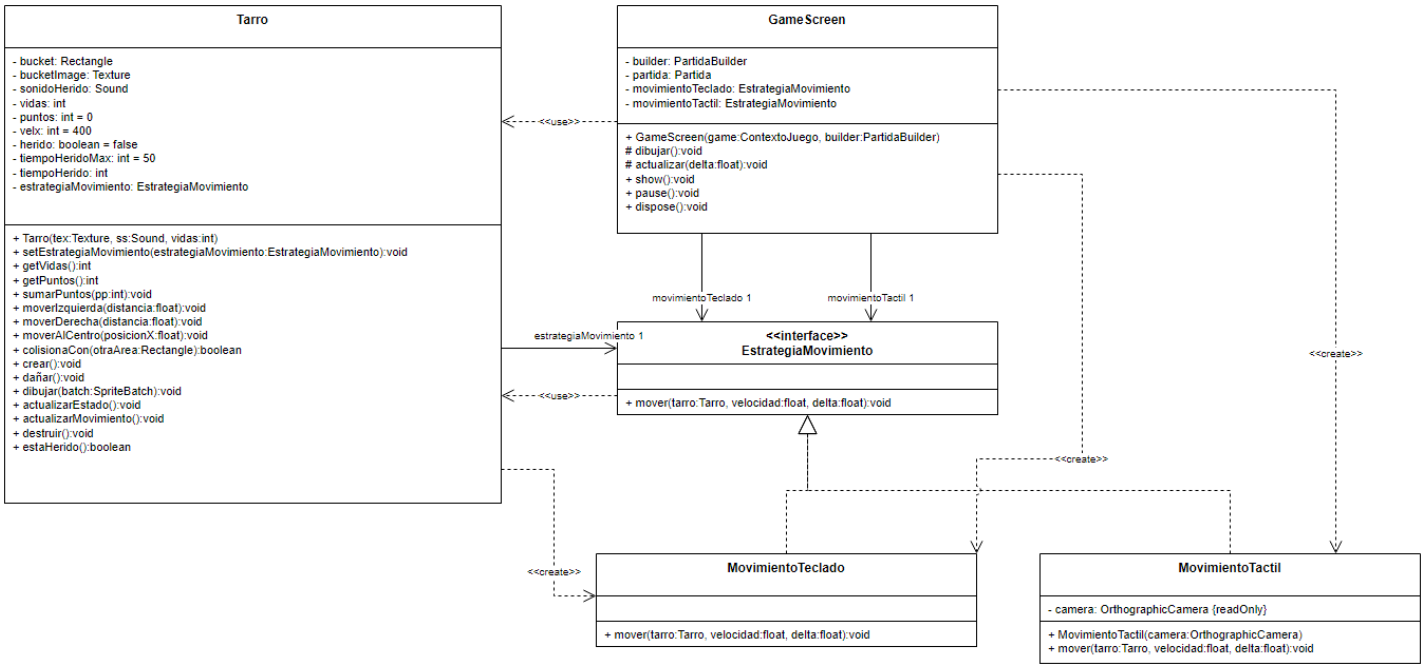
En la versión inicial, Tarro leía directamente las flechas del teclado en actualizarMovimiento. También tenía comentada una alternativa para mouse o toque, la cual dependía de la cámara de la pantalla. Para agregar otra forma de control era necesario modificar la lógica de movimiento de la clase.

Contexto

Nuestro juego permite mover un tarro horizontalmente para recoger gotas. Queríamos utilizar teclado o mouse sin cambiar el objeto que guarda las vidas, los puntos y la posición del jugador. Ambas formas de control realizan la misma operación, pero interpretan la entrada de manera diferente.

Solución

Dejamos el movimiento del tarro en dos estrategias que comparten el método mover. El tarro parte con MovimientoTeclado, que usa las flechas para calcular cuánto se desplaza y cuando el jugador presiona 2, GameScreen le asigna MovimientoTactil , que lo centra en la posición del puntero mientras se mantiene presionado el click izquierdo del mouse. Con la tecla 1 se vuelve al modo de teclado, como el cambio ocurre en el mismo tarro, se conservan las vidas y los puntos, y los límites de la pantalla se aplican con cualquiera de los dos controles. En el caso de que el método setEstrategiaMovimiento reciba null, conserva la estrategia que ya estaba activa.



GM-8 Aplicación del patrón de diseño Template Method

Requisito solicitado: Aplicación del patrón de diseño Template Method, describiendo problema, contexto, solución y UML.

Problema

Las pantallas del juego tienen que seguir un orden común para mostrarse: limpiar la pantalla, preparar la cámara, dibujar su contenido y actualizar lo que ocurre. Pero cada pantalla muestra cosas distintas y responde a controles diferentes. Si cada una tuviera que organizar todo el proceso por su cuenta, se repetirían los mismos pasos en varias clases.

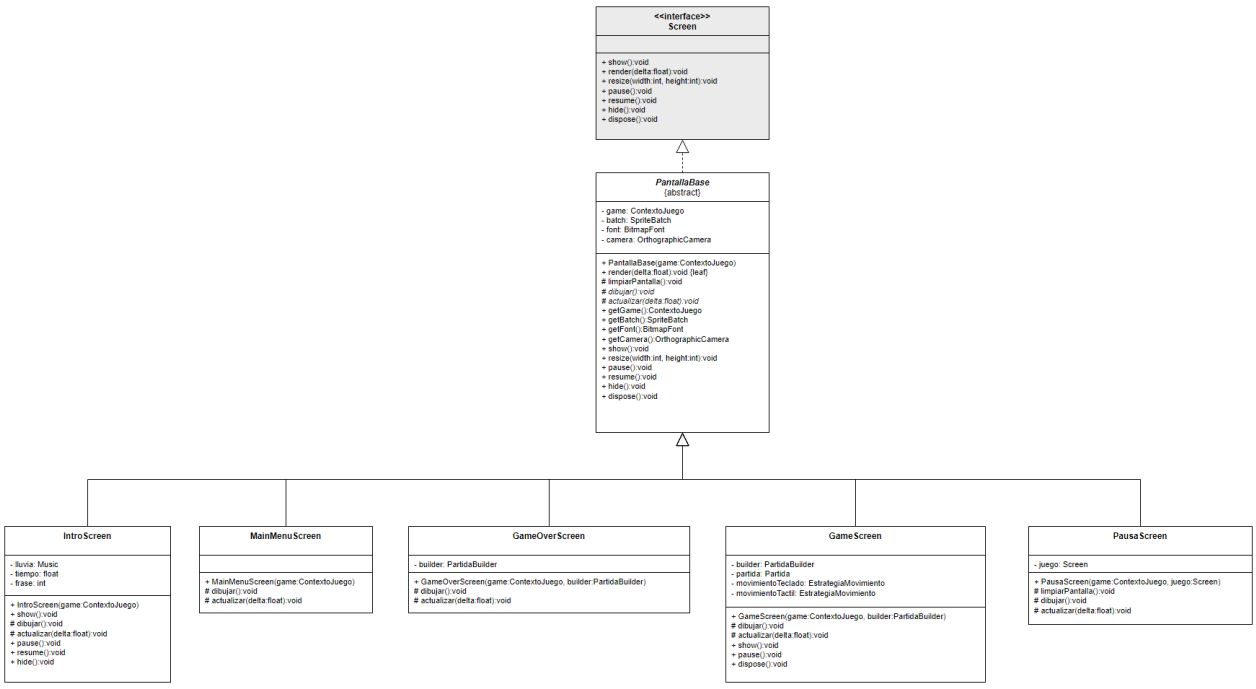
Contexto

El juego tiene una pantalla de introducción, un menú principal, una pantalla de partida, una pantalla de pausa y una pantalla de fin del juego. La introducción presenta la historia y permite saltar al menú con Enter. Cada una se comporta de forma distinta. En el menú se elige el modo fácil o difícil. Durante la partida se muestran el tarro, las gotas, el puntaje y las vidas. La pantalla de pausa permite volver a la partida, mientras que al terminar el juego se puede reiniciar en el mismo modo o volver al menú para elegir otra dificultad.

Solución

Como todas las pantallas necesitaban preparar la cámara antes de dibujar, dejamos esa secuencia en el método render de PantallaBase. El método es final y se encarga de limpiar la pantalla y preparar el dibujo, llama al método dibujar y, cuando termina, llama a actualizar. De esta manera, cada pantalla se ocupa de su propio contenido y de sus controles.

Por ejemplo, la introducción muestra la historia y pasa al menú cuando termina o cuando el jugador presiona Enter, mientras que la pantalla de pausa usa su propia versión de limpiarPantalla para cambiar el fondo y espera un click para retomar la partida.



GM-9 Aplicación del patrón de diseño Builder

Requisito solicitado: Aplicación del patrón de diseño Builder, describiendo problema, contexto, solución y UML.

Problema

Antes teníamos un solo builder que usaba modoDificil para decidir las vidas y el tiempo entre gotas. Además, la partida que entregaba quedaba incompleta, porque GameScreen todavía cargaba el fondo y asignaba el movimiento inicial del tarro. Queríamos que el director pudiera usar distintos builders y entregar una partida lista.

Contexto

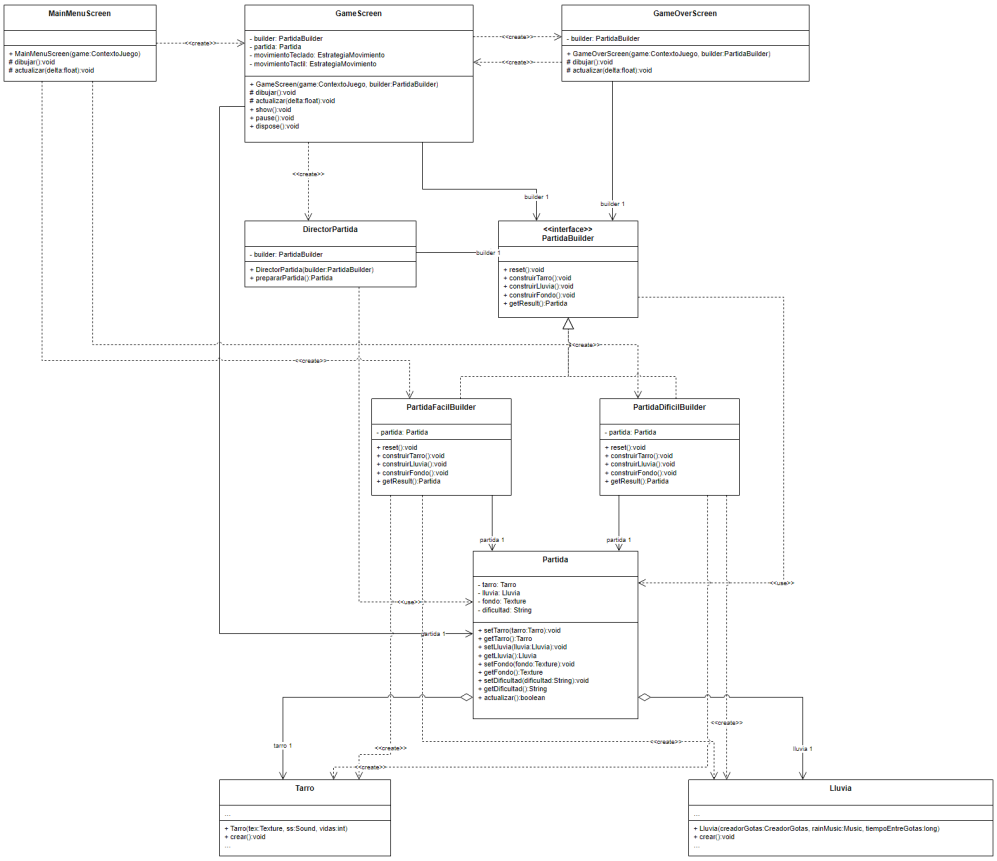
El jugador elige fácil o difícil en el menú. Ambos modos necesitan un tarro, una lluvia y un fondo, pero cada uno tiene sus propios valores. Esos elementos deben quedar preparados antes de comenzar.

Los modos se difieren en las vidas iniciales, el fondo y la frecuencia con que aparecen las gotas. Los dos comienzan con el teclado.

Solución

Cuando el jugador elige la dificultad, MainMenuScreen crea el builder de ese modo y se lo pasa a GameScreen, que le pide a DirectorPartida preparar la partida. El director sigue el mismo orden en ambos casos, empieza con una Partida nueva, le pide al builder que construya el tarro, la lluvia, el fondo y recupera el resultado con el método getResult.

Cada builder aporta los valores de su modo, de modo que en fácil el jugador parte con tres vidas, fondo azul y un intervalo de 0,3 segundos entre gotas, mientras que en difícil parte con dos vidas, fondo rojo y un intervalo de 0,1 segundos entre las gotas. En los dos casos se empieza con el teclado y si el jugador reinicia, se vuelve a usar el mismo builder para conservar la dificultad.



GM-10 Utilización de GitHub

Requisito solicitado: Utilización de GitHub, realizando al menos 6 commits.

Utilizacion

Utilizamos github para guardar y organizar el trabajo del grupo, donde cada integrante trabaja en su rama remota personal y la rama main mantiene la versión integrada del proyecto.

Las tareas se organizaron mediante issues, repartidos de forma equitativa y lógica entre integrantes, donde en conjunto se integran las partes al proyecto.

La rama main cumple con el requisito mínimo de 6 commits, con un total de 33 commits.

Repositorio

https://github.com/The-Nick-J/GM-Lluvia